

Region-Grounded Forced-Choice Attribute Verification for Multimodal Hallucination Detection

Prototype walkthrough, with step-by-step demo screenshots

Team	Register no.
Vanuj Gangrade	23BDS0282
Aryan Bhandari	23BDS0223
Harsh Parmeshwaram	23BDS0007

Guide: Dr. Gautam Majumdar | **Institution:** VIT Vellore

All screenshots in this document were produced by running the actual prototype on an RTX 3050 Laptop GPU (4 GB). Every number is taken from the project's result files.

1. Brief description of the innovation

AI models that describe images often **hallucinate**: they state things the picture does not show, such as calling a grey sky "sunny". Our system is a **hallucination detector**. Given a photograph and a claim about it, it decides whether the image supports the claim.

We target the hardest kind of claim, **attributes** ("is the sky *sunny* or *gloomy*?", "is the man *sitting* or *standing*?"). Published work (UNIHD, 2024) reports minimal progress on exactly this case and attributes it to the lack of a specialised attribute tool. This project builds that tool.

The two changes we make to the standard approach

	Standard approach	Our approach
Where the model looks	The whole photograph	Region grounding : find the object, crop it, judge only the crop
How the model decides	Scores one claim alone, then applies a hand-picked cutoff (threshold)	Forced choice : offers competing options (sunny vs gloomy) and picks the higher score. No cutoff to tune

In everyday terms: region grounding hands the examiner a magnifying glass; forced choice turns an essay question into a multiple-choice question. We test both changes separately, so we can say which one actually produces the improvement.

Headline result (held-out test set, opened once)

	Threshold decision	Forced choice
Whole image	A = 57.5% (standard baseline)	B = 77.9%
Cropped region	C = 59.5%	D = 80.1% (proposed)

D beats the baseline A by +22.6 percentage points, 95% confidence interval [+20.0, +25.1]. On the development set the gain was +22.3, so the effect replicated on unseen data.

2. Current status and prototype availability

Milestone	Content	Status
M0	Data, evaluation harness, unit tests, frozen splits	Complete
M1-M2	Baseline and proposed method on 100 pairs; go/no-go gate	Passed 2026-09-07
M3	All four cells on full development set, confidence intervals	Complete
M4	Existence, counting and relation modules; cost table; live demo	Complete
M5	Test split opened once; paper draft (IEEE two-column)	Complete

The prototype is working and available now

- **Runs locally** on a 4 GB laptop GPU (RTX 3050); peak memory 1.65 GiB. Also runs on free Colab/Kaggle GPUs; CPU fallback works.
- **Zero running cost:** two open-source Google models (OWLv2 and SigLIP, 358M parameters total). No large language model, no paid API.
- **Live demo command** accepts any image and a plain-English claim (Section 6).
- **Full reproducibility:** `python reproduce.py` re-runs every model in about 4 minutes and rebuilds every table.
- **241 automated tests**, all passing (Screenshot 1).
- Source code in a git repository, with a log of 51 recorded design decisions, a user guide (docs/GUIDE.md) and the paper source (docs/paper/main.tex).

3. Key features

- **Attribute verification** for state claims (colour, weather, size, etc.) and action claims (sit, stand, run, etc.).
- **Automatic claim parsing:** a sentence like "the man is sitting" is split into object = *man*, attribute = *sit*.
- **Object localisation and cropping** with a saved crop image, so the user can see what the model looked at.
- **Verdict with confidence:** SUPPORTED / NOT SUPPORTED, plus the score of each option.
- **Complete checker suite:** existence ("is there a dog?"), counting ("are there three people?") and relation ("is the person touching the grass?") modules.
- **Honest failure handling:** if the object cannot be found, the answer is flagged (3.4% of questions) rather than hidden.
- **Memory-efficient staging:** the two models are never in GPU memory at the same time.
- **Statistically sound evaluation:** 10,000-resample bootstrap confidence intervals over images; test data locked behind an environment variable and opened exactly once.

4. Novelty

Our novelty is:

(a) Region grounding followed by forced choice, as a dedicated attribute-verification tool

Our method combines two steps into one tool built specifically for attribute claims. **First, region grounding:** the object named in the claim is located and cropped, so the model judges only the object and not the rest of the photograph. **Then, forced choice:** on that crop, the claimed attribute is compared head-to-head with a competing attribute (sunny vs gloomy, sit vs stand) and the higher score wins. There is no hand-tuned cutoff anywhere in the decision.

This fills the gap that UNIHD (2024) identified: existing detectors have no specialised tool for attribute claims. Our combined method (Cell D) reaches **80.1%** on the held-out test set against 57.5% for the standard approach, a gain of **+22.6 points**, using only two small open models on a 4 GB laptop GPU.

(b) A controlled 2x2 experiment that separates the two changes

We built four versions so each change is measured on its own: A (neither), B (forced choice only), C (cropping only), D (both). Without B and C nobody can tell which change caused the gain.

(c) A counter-intuitive finding about where the gain comes from

What is isolated	Gain on test	Verdict
Forced choice alone (B - A)	+20.4 points	Large, clearly significant
Region grounding alone (C - A)	+2.0 points	Barely significant (lower bound +0.1)
Cropping once forced choice is used (D - B)	+2.3 points	Not significant

The field commonly assumes that zooming in on the object is what fixes attribute errors. Our data shows the **decision rule** (choosing between options rather than grading one claim) carries almost all of the effect. The gain also survives when the competing option comes from an antonym list written before any results existed (Cell D-ext: +19.0 points over baseline), so it is not an artefact of the benchmark supplying the pairs.

(d) Three flaws discovered in the AMBER benchmark

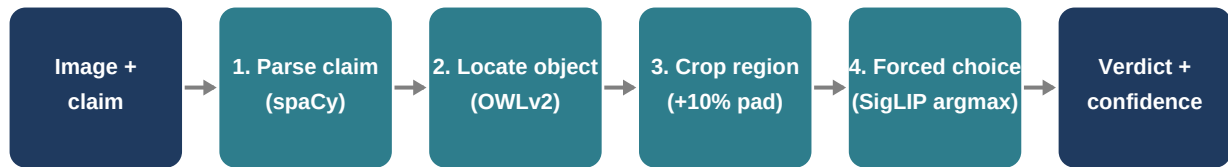
Flaw	Cheat that scores 100% without looking at the image
In all 2,774 attribute pairs, the true answer has the lower question id	Answer "yes" to the lower id
All 4,924 existence questions ask about absent objects	Always answer "no"
Relation answers are constant per type (975 all-yes, 689 all-no)	Answer by type

We implemented the first cheat, labelled it as a benchmark artifact in our results, and showed that the question types our result depends on are free of all three flaws.

(e) Integrity by design

Scoring code was written and tested before any model was connected, with a second independent implementation cross-checked on 20,000 random cases. No threshold, prompt or constant was ever tuned to improve a reported number. Two of our own early claims were later disproved by more data, and both withdrawals are reported openly.

5. Prototype model



Models are loaded one at a time: OWLv2 runs, is freed, then SigLIP loads -> peak 1.65 GiB VRAM
Evaluation: the competing option comes from AMBER's matched pair. Demo: from configs/antonyms.yaml.

Figure 1. Pipeline of the prototype.

Component	Model / tool	Role	Size
Claim parser	spaCy en_core_web_sm	Splits the sentence into object and attribute	small
Detector	google/owlv2-base-patch16-ensemble	Finds the object; prompt "a photo of a {object}"; keeps boxes with score ≥ 0.10 ; removes duplicates (NMS, IoU 0.5)	155 M params
Scorer	google/siglip-base-patch16-224	Scores "a photo of a {attribute} {object}" for each option against the crop	203 M params
Total		No LLM, no paid API, API cost = 0	358 M params

The four experimental cells

Cell	Looks at	Decides by	Test accuracy
A (baseline)	Whole image	Threshold (cutoff fitted on development data)	57.5%
B	Whole image	Forced choice	77.9%
C	Cropped region	Threshold	59.5%
D (proposed)	Cropped region	Forced choice	80.1%

Cropping details: the best box is padded by 10% on each side, clipped to the image, and enlarged to at least 64x64 pixels. The prompt wording is identical in every cell, and options are always sorted alphabetically so their order can never reveal the answer.

6. Step-by-step demo walkthrough

Step 1 - Verify the installation: run the test suite

Before anything else we confirm the code is correct. The suite checks data counts, pair construction, splits, both independent accuracy implementations, the detector's cropping, and the four cells. No GPU is needed.

```
Anaconda Prompt - (capstone)
(capstone) C:\...\capstone> python -m pytest -q
..... [ 29%]
..... [ 59%]
..... [ 89%]
..... [100%]
241 passed in 26.43s
```

Screenshot 1. All 241 automated tests pass.

Step 2 - Choose an input image and write a claim

We use AMBER_1.jpg from the benchmark and the claim **"the sky is sunny"**. The command is:

```
python -m src.demo --image data\images\AMBER_1.jpg --claim "the sky is sunny"
```



Screenshot 2. The input photograph (AMBER_1.jpg).

Step 3 - Parse the claim

spaCy splits the sentence into its parts: object = **sky**, attribute = **sunny**, type = **state**. The competing option, **gloomy**, is looked up in configs/antonyms.yaml. (During evaluation the competitor comes from the benchmark's matched pair instead.)

Step 4 - Locate the object (OWLv2)

The detector is asked for "a photo of a sky" and returns a box with score 0.42. Red = best detection, blue = the padded crop area.



Screenshot 3. Left: detected sky (score 0.42). Right: the crop saved to results/demo/AMBER_1_sky.jpg.

Step 5 - Forced choice on the crop (SigLIP)

OWLv2 is released from memory and SigLIP is loaded. It scores "a photo of a gloomy sky" and "a photo of a sunny sky" against the crop only. The higher score wins.

Step 6 - Read the verdict

Anaconda Prompt - (capstone)

```
(capstone) C:\...\capstone> set HF_HOME=C:\hf
(capstone) C:\...\capstone> python -m src.demo --image data\images\AMBER_1.jpg --claim "the sky is sunny"
Claim:      the sky is sunny
Parsed:     object='sky' attribute='sunny' (state)
Object:     sky (detector score 0.42)
Crop saved: results\demo\AMBER_1_sky.jpg
Options:    gloomy 0.00 | sunny 1.00
Verdict:    SUPPORTED (confidence 1.00)
Note:       confidence is uncalibrated (D-011); the contrast came from the demo antonym map,
            never from the evaluation path (TRD §13).
```

Screenshot 4. Actual output: sunny 1.00 vs gloomy 0.00, verdict SUPPORTED.

Step 7 - Test a false claim on the same image

Now the claim is **"the sky is gloomy"**. Same crop and same two options, but the claim names the losing option, so the system flags it as a hallucination.

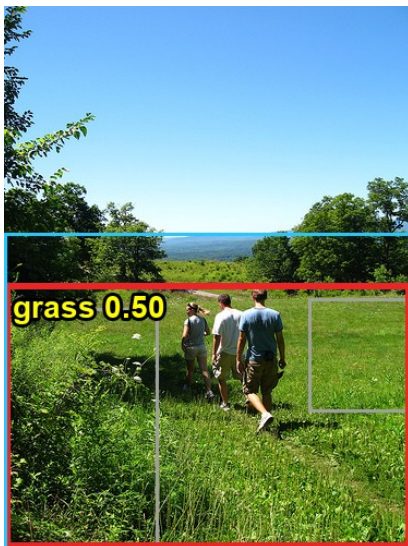
Anaconda Prompt - (capstone)

```
(capstone) C:\...\capstone> set HF_HOME=C:\hf
(capstone) C:\...\capstone> python -m src.demo --image data\images\AMBER_1.jpg --claim "the sky is gloomy"
Claim:      the sky is gloomy
Parsed:     object='sky' attribute='gloomy' (state)
Object:     sky (detector score 0.42)
Crop saved: results\demo\AMBER_1_sky.jpg
Options:    gloomy 0.00 | sunny 1.00
Verdict:    NOT SUPPORTED (confidence 1.00)
Note:       confidence is uncalibrated (D-011); the contrast came from the demo antonym map,
            never from the evaluation path (TRD §13).
```

Screenshot 5. The false claim is correctly rejected: NOT SUPPORTED.

Step 8 - A different object in the same image

Claim: **"the grass is green"**. The detector finds three grass regions (scores 0.50, 0.12, 0.10; weaker ones in grey); the best one is cropped. Competitor: brown.



Screenshot 6. Left: grass detections (best in red). Right: the crop used for the decision.

Anaconda Prompt - (capstone)

```
(capstone) C:\...\capstone> set HF_HOME=C:\hf
(capstone) C:\...\capstone> python -m src.demo --image data\images\AMBER_1.jpg --claim "the grass is green"
Claim:      the grass is green
Parsed:     object='grass' attribute='green' (state)
Object:     grass (detector score 0.50)
Crop saved: results\demo\AMBER_1_grass.jpg
Options:    brown 0.01 | green 0.99
Verdict:    SUPPORTED (confidence 0.99)
Note:       confidence is uncalibrated (D-011); the contrast came from the demo antonym map,
            never from the evaluation path (TRD §13).
```

Screenshot 7. green 0.99 vs brown 0.01, verdict SUPPORTED.

Step 9 - An action claim

Claim on AMBER_2.jpg: **"the man is sitting"**. The parser recognises an action and reduces "sitting" to its base form **sit**; the competitor is **stand**. The man in the boat is located and cropped.



Screenshot 8. Left: man detected (score 0.43). Right: the crop.

```
Anaconda Prompt - (capstone)
(capstone) C:\...\capstone> set HF_HOME=C:\hf
(capstone) C:\...\capstone> python -m src.demo --image data\images\AMBER_2.jpg --claim "the man is sitting"
Claim:      the man is sitting
Parsed:     object='man' attribute='sit' (action)
Object:     man (detector score 0.43)
Crop saved: results\demo\AMBER_2_man.jpg
Options:    sit 0.98 | stand 0.02
Verdict:    SUPPORTED (confidence 0.98)
Note:      confidence is uncalibrated (D-011); the contrast came from the demo antonym map,
           never from the evaluation path (TRD §13).
```

Screenshot 9. sit 0.98 vs stand 0.02, verdict SUPPORTED.

Note on confidence: the values are relative option scores, not calibrated probabilities; the project does not claim calibration.

Step 10 - The full evaluation on the held-out test set

The demo shows single examples. The real measurement runs all four cells over 1,690 test questions on 296 unseen images, with every threshold frozen from the development set. The script below prints the result from the stored predictions, so no GPU is needed.

Anaconda Prompt - (capstone)

```
(capstone) C:\...\capstone> python scripts\test_report.py
cropped region      | C  0.5953          | D  0.8012
controls: A' 0.5775   C' 0.6012
D-ext (external contrast, n=1589): 0.7602

PER CELL
A   acc=0.5751 CI[0.5610,0.5898] yes=923/845 tau=-11.027273654937744
B   acc=0.7787 CI[0.7512,0.8054] yes=845/845 tau=NOT_APPLICABLE
C   acc=0.5953 CI[0.5797,0.6111] yes=589/845 tau=-5.086052179336548
D   acc=0.8012 CI[0.7742,0.8276] yes=845/845 tau=NOT_APPLICABLE
Ap  acc=0.5775 CI[0.5638,0.5914] yes=859/845 tau=-10.547151565551758
Cp  acc=0.6012 CI[0.5878,0.6148] yes=877/845 tau=-7.020545482635498
Dext acc=0.7602 CI[0.7346,0.7857] yes=861/792 tau=NOT_APPLICABLE

ATtribution - paired bootstrap over test images, 10,000 resamples
C - A   region grounding alone           +0.0201 [+0.0012, +0.0391] EXCLUDES ZERO
B - A   forced choice alone              +0.2036 [+0.1781, +0.2288] EXCLUDES ZERO
B - Ap  forced choice, net of base rate  +0.2012 [+0.1756, +0.2269] EXCLUDES ZERO
D - C   forced choice given cropping      +0.2059 [+0.1808, +0.2306] EXCLUDES ZERO
D - Cp  forced choice given cropping, net +0.2000 [+0.1736, +0.2259] EXCLUDES ZERO
D - B   cropping given forced choice      +0.0225 [-0.0012, +0.0462] includes zero
D - A   HEADLINE: both                   +0.2260 [+0.2001, +0.2514] EXCLUDES ZERO

D-ext MATCHED SUBSET coverage 1589/1690 = 0.9402
D   on covered  0.8030
D-ext on covered 0.7602
A   on covered  0.5702
D-ext - A (covered) +0.1901 [+0.1645, +0.2157] EXCLUDES ZERO
D-ext - D (covered) -0.0428 [-0.0589, -0.0260] EXCLUDES ZERO
```

Screenshot 10. Output of python scripts\test_report.py. Green: significant; amber: not significant.

7. Findings

- **The proposed method works:** 80.1% vs 57.5% for the standard baseline, +22.6 points, confidence interval [+20.0, +25.1]. Development and test agree closely (+22.3 vs +22.6).
- **Forced choice is the mechanism:** it alone gives +20.4 points. Region grounding alone gives only +2.0 points, and its interval clears zero by just 0.1 points.
- **An early claimed interaction did not replicate:** on development the two changes seemed to boost each other (+2.2); on test this dropped to +0.2. We report it as not replicated.
- **The gain is not a benchmark shortcut:** with externally chosen competitors (D-ext) the method still beats the baseline by +19.0 points.
- **The benchmark has three construction flaws**, each allowing a perfect score without looking at the image; our result avoids all three.
- **Other modules (development set):** counting 72.0% accuracy; existence false-alarm rate 12.4% at detector threshold 0.10 (2.1% at 0.30); relation reported per type because each type has a constant answer.
- **Cost:** about 32 ms per attribute question, 1.65 GiB peak GPU memory, zero API cost.

Limitations

- Region grounding's small effect may not hold on a different data split.
- Forced choice needs a competing option; about 21% of action verbs (swim, jump, surf) have no natural English opposite.
- In 3.4% of questions the object was not detected and the whole image was used.
- Confidence scores are uncalibrated.
- English only, AMBER's 340-object vocabulary, zero-shot (no training).

8. Future scope

- **Replicate region grounding** on other benchmarks and splits to settle whether its small effect is real.
- **Automatic competitor generation** for attributes with no natural opposite, so forced choice applies to every claim.
- **Calibrate confidences** so the score can be read as a real probability.
- **Improve the detector fallback** for small or unusual objects.
- **Free-form captions:** mark the exact hallucinated words in a full AI-generated description (AMBER's generative half).
- **Plug into full pipelines** such as Woodpecker or UNIHD as their attribute tool and measure the end-to-end gain.
- **Beyond English** and beyond a fixed object vocabulary; scene-text (OCR) claims.
- **Use forced choice as a training signal** to reduce hallucination in the vision-language model itself.

Data: AMBER benchmark (Wang et al., 2023), Apache-2.0. Models: OWLv2 and SigLIP (Google), used zero-shot at pinned revisions.